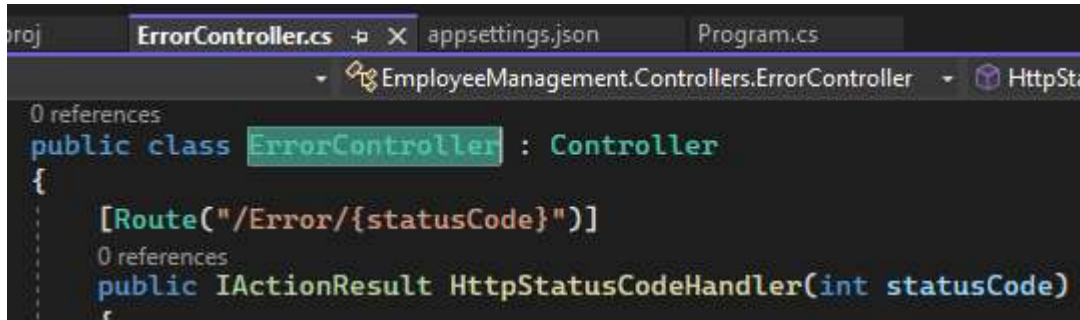


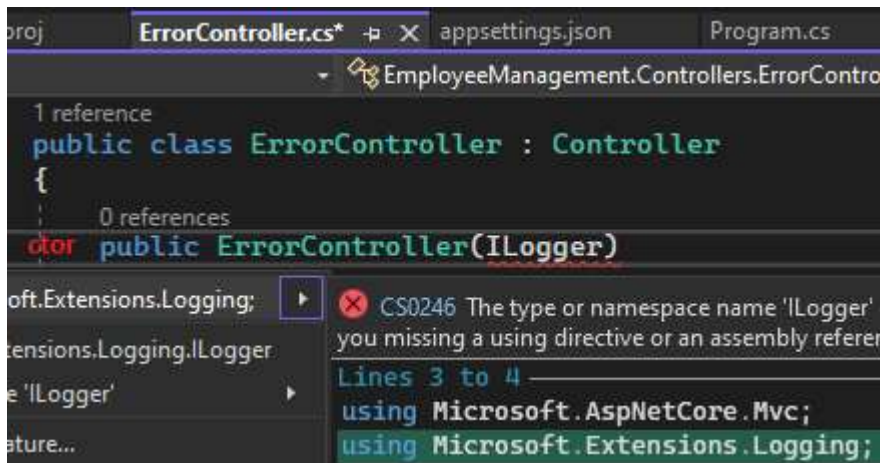
62 - Logging exceptions

В CMD виждаме много информация, а в Debug mode, стартиран от VS – output виждаме също, но за по-голяма пълнота въвеждаме допълнителен код. Към момента, грешките се прихващат от ErrorController:



```
public class ErrorController : Controller
{
    [Route("/Error/{statusCode}")]
    public IActionResult HttpStatusCodeHandler(int statusCode)
    {
    }
```

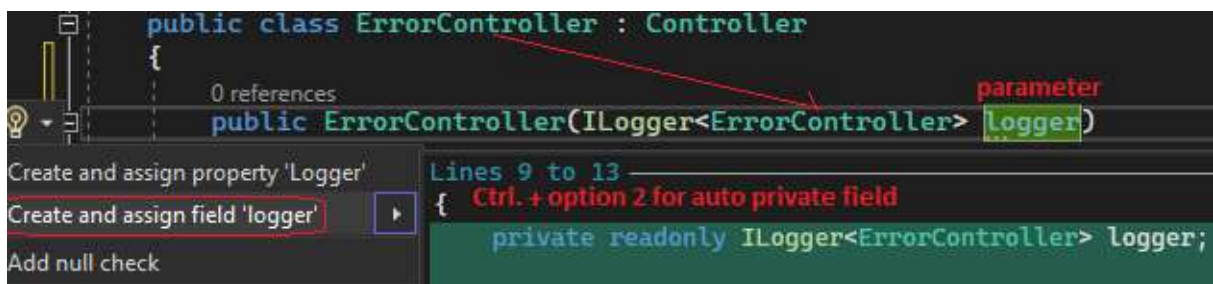
Ако има изключение, ние искаме информация за него. За целта първо трябва да вмъкнем ILoggerInterface, предоставен за целта. С него ще може и да групираме съобщенията за грешки.



```
public ErrorController(ILogger logger)
{
}
```

CS0246 The type or namespace name 'ILogger' you missing a using directive or an assembly reference

using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;



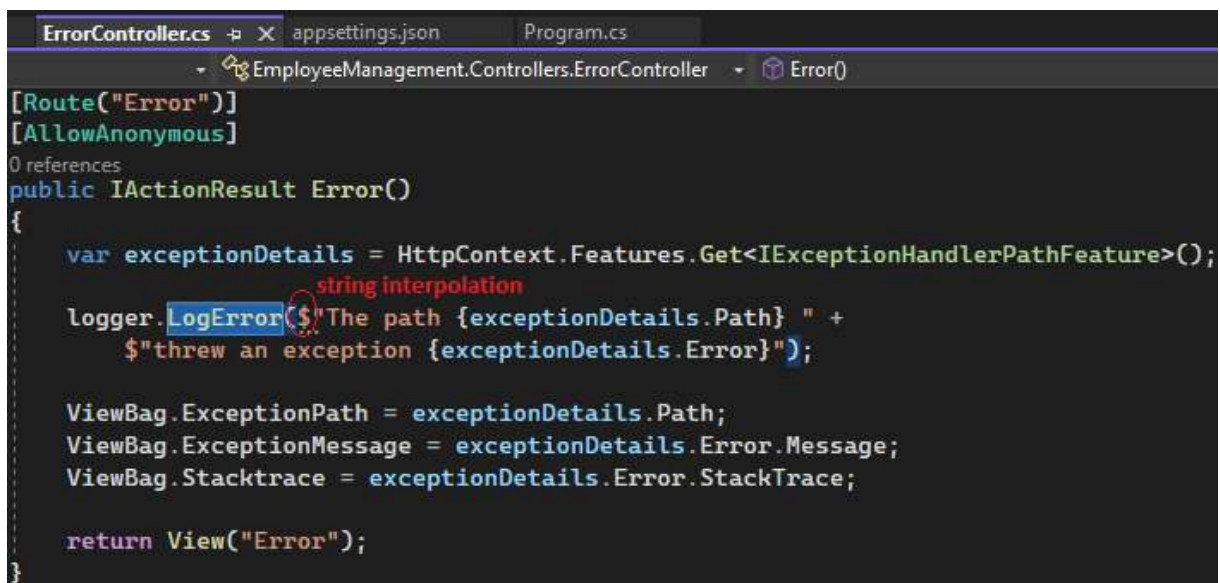
```
public ErrorController(ILogger<ErrorController> logger)
{
    private readonly ILogger<ErrorController> logger;
```

Вече тази нова променлива от Private field може да се използва навсякъде в кода по-надолу.

```
public class ErrorController : Controller
{
    private readonly ILogger<ErrorController> logger;

    0 references
    public ErrorController(ILogger<ErrorController> logger)
    {
        this.logger = logger;
    }
}
```

```
public ErrorController(ILogger<ErrorController> logger)
{
    this.logger = logger;
}
```



```
EmployeeManagement.Controllers.ErrorController Error()
[Route("Error")]
[AllowAnonymous]
0 references
public IActionResult Error()
{
    var exceptionDetails = HttpContext.Features.Get<IExceptionHandlerPathFeature>();

    logger.LogError($"The path {exceptionDetails.Path} " +
        $"threw an exception {exceptionDetails.Error}");

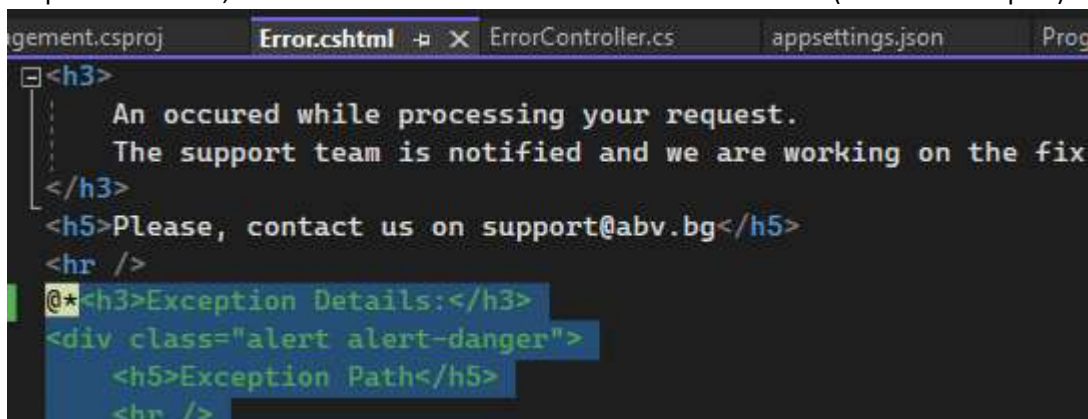
    ViewBag.ExceptionPath = exceptionDetails.Path;
    ViewBag.ExceptionMessage = exceptionDetails.Error.Message;
    ViewBag.Stacktrace = exceptionDetails.Error.StackTrace;

    return View("Error");
}
```

Използваме го в Error action, където включваме \$ = string interpolation, вмъкване на стринг в съобщението за грешка, където параметрите са вкарани {между}. При нов ред използваме + \$.

```
logger.LogError($"The path {exceptionDetails.Path} " +
    $"threw an exception {exceptionDetails.Error}");
```

В същото време, вече искаме тези съобщения да се изведат автоматично на екрана, вместо стария View Error, затова отиваме в папка Shared и ги изчистваме (аз закоментирам).



```
management.csproj Error.cshtml ErrorController.cs appsettings.json Prog
<h3>
    An occurred while processing your request.
    The support team is notified and we are working on the fix
</h3>
<h5>Please, contact us on support@abv.bg</h5>
<hr />
@* <h3>Exception Details:</h3>
<div class="alert alert-danger">
    <h5>Exception Path</h5>
    <hr />
```

В ErrorController имаме по-нагоре още един метод, който отговаря за грешен URL с 404, но вместо информация за грешката, искаме да я регистрираме, така че отново закоментираме:

```
csproj | Error.cshtml | ErrorController.cs | appsettings.json | Program.cs
EmployeeManagement.Controllers.ErrorController | HttpStatusCodeHandler(int statusCode)
[Route("/Error/{statusCode}")]
0 references
public IActionResult HttpStatusCodeHandler(int statusCode)
{
    var statusCodeResult = HttpContext.Features.Get<IStatusCodeReExecuteFeature>();

    switch (statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Sorry, the resource you requested could not be found";
            logger.LogWarning($"404 Error Occured. Path = {statusCodeResult.OriginalPath}" +
                $"and QueryString = {statusCodeResult.OriginalQueryString}");
            //ViewBag.Path = statusCodeResult.OriginalPath;
            //ViewBag.QS = statusCodeResult.OriginalQueryString;
            break;
    }

    return View("NotFound");
}
```

```
logger.LogWarning($"404 Error Occured. Path = {statusCodeResult.OriginalPath}" +
    $"and QueryString = {statusCodeResult.OriginalQueryString}");
```

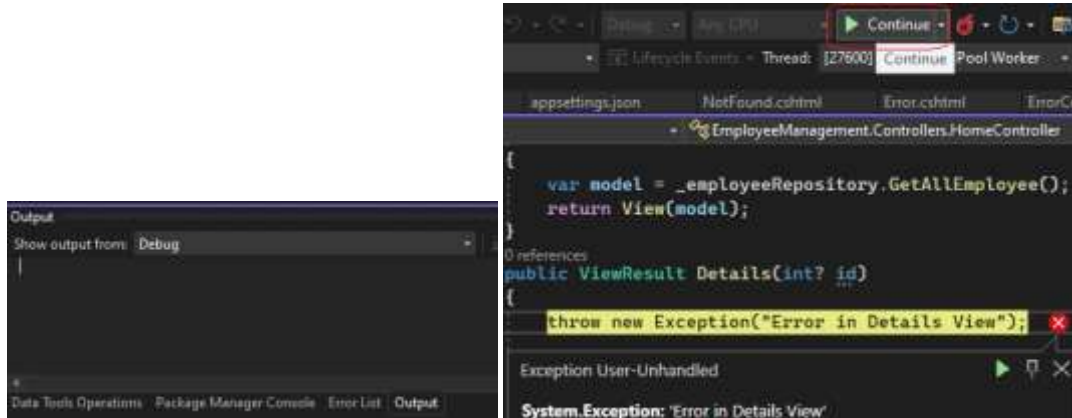
Последно изчистваме NotFound View в папка Shared:

```
ment.csproj | NotFound.cshtml | Error.cshtml | ErrorController.cs
@model int
@{
    ViewBag.PageTitle = "Not Found";
}
<h1>@ViewBag.ErrorMessage</h1>
@*<h1>@ViewBag.Path</h1>
<h1>@ViewBag.QS</h1>*@
<a asp-action="index" asp-controller="home">
    Click here to navigate to the home page
</a>
```

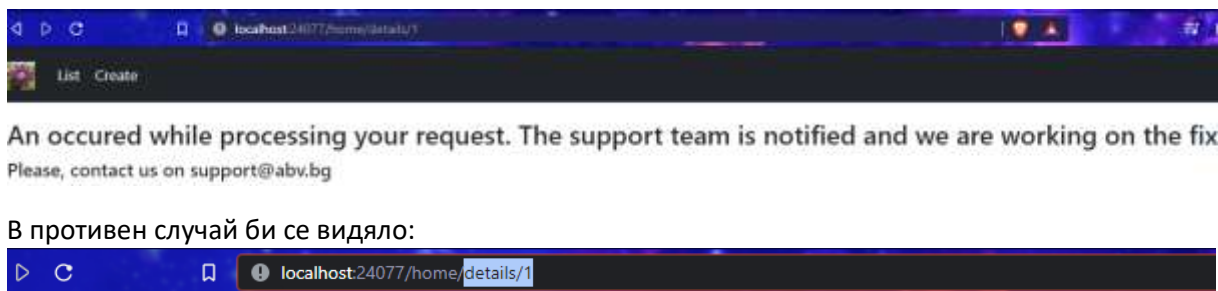
```
ent.csproj | appsettings.json | NotFound.cshtml | ErrorController.cs
n.schemastore.org/appsettings.json
"Logging": {
  "LogLevel": {
    "Default": "Warning",
    "Microsoft": "Information"
  }
}
```

Когато стартираме в Debug, отново имаме прекалено много съобщения, започващи с Microsoft, затова, за да оставим само предупрежденията, Microsoft: Information -> Warning

След Debug вече нямаме никакви видими съобщения



Където имаме нова възможност за старт: Continue:



HTTP Error 502.3 - Bad Gateway

The connection with the server was terminated abnormally

Most likely causes:

- The CGI application did not return a valid set of HTTP errors.
- A server acting as a proxy or gateway was unable to process the request due to an error in a parent gateway.

Things you can try:

- Use DebugDiag to troubleshoot the CGI application.
- Determine if a proxy or gateway is responsible for this error.

Detailed Error Information:

Module AspNetCoreModuleV2
Notification ExecuteRequestHandler
Handler aspNetCore
Error Code 0x80072efe

Requested URL http://localhost:24077/home/details/1
Physical Path D:\Exercises\EmployeeManagement\home\details\1
Logon Method Anonymous
Logon User Anonymous

Виждаме:


```
public class ErrorController : Controller
{
    private readonly ILogger<ErrorController> logger;

    public ErrorController(ILogger<ErrorController> logger)
    {
    }

    [Route("/Error/{statusCode}")]
    public IActionResult HttpStatusCodeHandler(int statusCode)
    {
    }

    [Route("Error")]
    [AllowAnonymous]
    public IActionResult Error()
    {
        var exceptionDetails = HttpContext.Features.Get<IExceptionHandlerDetails>();

        logger.LogError($"The path {exceptionDetails.Path} " +
            $"threw an exception {exceptionDetails.Error}");
    }
}
```

Debug

Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware: Error: The path /foo/bar/1 threw an exception System.InvalidOperationException: Sequence contains no matching element

Още един опит при несъществуващ адрес с променливи:



Sorry, the resource you requested could not be found

[Click here to navigate to the home page](#)

Където в Debug извежда 1 предупреждение:

Output

Show output from: Debug

ErrorController: Warning: 404 Error Occured. Path = /foo/bar and QueryString = ?email=support@abv.bg&abc=xyz

Logging Exceptions in ASP.NET Core

- Inject an instance of **ILogger**
- Specify the **type of the Controller** into which **ILogger** is injected as the generic argument

```
public class ErrorController : Controller
{
    private readonly ILogger<ErrorController> logger;

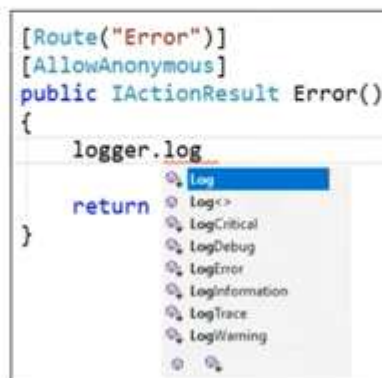
    public ErrorController(ILogger<ErrorController> logger)
    {
        this.logger = logger;
    }

    // Rest of the code
}
```

Logging Exceptions in ASP.NET Core

- Use the injected **ILogger** instance methods to log messages
 - **LogInformation**
 - **LogWarning**
 - **LogError**
 - **Etc....**

```
[Route("Error")]
[AllowAnonymous]
public IActionResult Error()
{
    logger.log
    return
}
```



Next : Logging to a file